

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	KS THARUNESHWAR	Reg. No.:	192572148
Section / Batch:	2025	Date:	29-08-2026

Instructions to Students

- Answer ALL questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q3	Q1 –40 Q2 –30 Q3 -30
GRAND TOTAL:		100 Marks	

SECTION A – Pseudocode Writing Task

Q1. N-Queens Problem Design a pseudocode using Backtracking to solve the N-Queens problem for an $N \times N$ chessboard. Clearly identify inputs (board size) and expected output (valid queen placements). Develop a logically correct algorithm that ensures no two queens attack each other. Use appropriate control structures such as loops and conditional checks for row, column, and diagonal safety. Ensure the pseudocode is well-structured and easy to follow. Handle all possible configurations and optimize by pruning invalid placements.

Problem Definition

The N-Queens problem is a classic combinatorial constraint satisfaction problem. The goal is to place N non-attacking queens on an $N \times N$ chessboard such that no two queens share the same row, column, or diagonal.

Input: N (an integer representing board dimensions $N \times N$ and number of queens). **Output:** A 2D matrix or a 1D placement array `board[0..N-1]` representing valid queen locations, or an indication that no solution exists.

Algorithmic Approach

We use Backtracking with a Depth-First Search (DFS) strategy. Queens are placed row-by-row from top to bottom (row 0 to $N-1$). For each row, we attempt to place a queen in column c (from 0 to $N-1$). Before placing, we execute safety checks against previously placed queens. If safe, we recursively move to the next row. If placing a queen leads to a dead end (no valid column in subsequent rows), we backtrack by removing the queen from the current row and trying the next column.

State-Space Representation

The state-space tree is an N -ary tree of depth $N + 1$ (including root at depth 0). Each node at depth 'row' represents a partial assignment of queens to rows 0 through row-1. Branching factor at each node is N (representing possible column choices 0 to $N-1$). Pruning terminates subtrees whenever a column or diagonal collision occurs.

Pseudocode

```
Algorithm SolveNQueens(N):
    Input: Board size N
    Output: Solution matrix / array or indication of failure

    Initialize board[0..N-1] with -1 // board[r] stores column index for queen in row r
    if NQueensRecursive(board, 0, N) == TRUE:      PrintBoard(board, N)
    return TRUE  else:
        Print "No solution exists"
    return FALSE

Algorithm NQueensRecursive(board, row, N):
    // Base Case: All N queens are successfully placed
    if row == N:
        return TRUE

    // Try placing queen in each column of the current row
    for col = 0 to N - 1:      if IsSafe(board, row, col, N) ==
    TRUE:
        board[row] = col      // Place queen at (row, col)

        // Recursive Call: Place queen in next row
        if NQueensRecursive(board, row + 1, N) == TRUE:
            return TRUE

        // Backtracking Step: Undo choice
        board[row] = -1

    return FALSE // Backtrack to previous row

Algorithm IsSafe(board, row, col, N):
    for i = 0 to row - 1:      prevCol =
    board[i]

    // 1. Column Check
    if prevCol == col:

        return FALSE

    // 2. Left Diagonal Check (row - col is equal)
    if (i - prevCol) == (row - col):      return FALSE

    // 3. Right Diagonal Check (row + col is equal)
    if (i + prevCol) == (row + col):      return FALSE

    return TRUE
```

Logical Explanation

1. Row-wise placement ensures that no two queens ever occupy the same row by construction.

2. Column safety check ensures no previous row i has $\text{board}[i] == \text{col}$.
3. Diagonal safety check relies on geometric properties: elements on the same main diagonal (left-to-right) have equal $(\text{row} - \text{col})$ difference, while elements on the anti-diagonal (right-to-left) have equal $(\text{row} + \text{col})$ sum.
4. Recursion proceeds to $\text{row} + 1$ upon finding a safe position. If a downstream call fails, backtracking sets $\text{board}[\text{row}] = -1$ and tests the next column.

Pruning Conditions

Pruning occurs instantly in `IsSafe()` before initiating a recursive call:

- Column Collision: $\text{board}[i] == \text{col}$
- Left Diagonal Collision: $(i - \text{board}[i]) == (\text{row} - \text{col})$
- Right Diagonal Collision: $(i + \text{board}[i]) == (\text{row} + \text{col})$

Invalid branches are immediately killed, preventing exploration of full subtrees.

Complexity Analysis

Worst-case Time Complexity: $O(N!)$ — In the worst case, searching through valid queen arrangements examines N choices for row 0, $N-1$ for row 1, $N-2$ for row 2, giving $N!$ operations bounded above.

Space Complexity: $O(N^2)$ for explicit grid representation, or $O(N)$ using a 1D position array $\text{board}[N] + O(N)$ recursion stack depth.

Q2. Subset Sum Problem Write a pseudocode to determine whether a subset of given integers sums to a target value using Backtracking. Clearly define inputs (set of numbers and target sum) and output (true/false or subset). Design a complete algorithm with proper branching and pruning conditions. Use control structures effectively to explore subsets. Present the pseudocode in a structured and readable format. Ensure the solution handles all cases and avoids unnecessary computations.

Problem Definition

Given a set/array of integers S and a target integer sum target , the Subset Sum Problem seeks to determine whether there exists a subset of elements whose sum equals target , and if so, return the subset.

Input: Array $S[0..n-1]$ of integers and target integer target .

Output: TRUE alongside the chosen subset if a solution exists; FALSE otherwise.

Algorithmic Approach

We construct a binary decision tree using recursive backtracking. At each index index , we make two branching decisions: (1) Include element $S[\text{index}]$ in the current subset, or (2) Exclude element $S[\text{index}]$. We evaluate currentSum against target to terminate early or backtrack when conditions are violated.

State-Space Representation

A binary state-space tree of max depth n . Every node at level i represents a choice regarding element $S[i]$. Left branches correspond to 'Include $S[i]$ ', right branches to 'Exclude $S[i]$ '. Leaves represent complete subset evaluations.

Pseudocode

```
Algorithm SolveSubsetSum(S, target):
    Input: Array S of size n, target sum
    Output: Boolean and prints valid subset

    n = length(S)
    subset = empty array/list
    return SubsetSumRecursive(S, n, 0, 0, target, subset)

Algorithm SubsetSumRecursive(S, n, index, currentSum, target, subset):
    // Target Found    if
currentSum == target:
    Print "Subset found: ", subset
    return TRUE

    // Base Cases for Termination
    if index == n or currentSum > target: // Pruning assumes non-negative integers
return FALSE

    // Branch 1: Include S[index]    subset.append(S[index])    if
SubsetSumRecursive(S, n, index + 1, currentSum + S[index], target, subset) ==
TRUE:
    return TRUE

    // Backtracking Step: Exclude S[index]
subset.pop() // Undo choice

    // Branch 2: Exclude S[index]    if SubsetSumRecursive(S, n, index + 1,
currentSum, target, subset) == TRUE:
    return TRUE

    return FALSE
```

Logical Explanation

1. Inclusion Branch: Adds $S[index]$ to currentSum and subset list, advancing index to index + 1.
2. Exclusion Branch: Pops $S[index]$ from subset, leaving currentSum unchanged, advancing index to index + 1.
3. Target Checking: Returns TRUE immediately upon reaching $currentSum == target$.
4. Backtracking: If including $S[index]$ yields no solution, popping it restores state before exploring the exclusion branch.

Pruning Conditions

- Over-Sum Pruning: If $currentSum > target$, the algorithm prunes the current subtree immediately.
- CRITICAL ASSUMPTION: This pruning condition ($currentSum > target$) is strictly valid ONLY when all elements in array S are non-negative. If S contains negative numbers,

adding subsequent elements could reduce currentSum back toward target, making pruning unsafe.

Complexity Analysis

Worst-case Time Complexity: $O(2^n)$ — In the worst-case scenario (e.g., no subset equals target), the algorithm explores all 2^n possible subsets of an n -element set.

Space Complexity: $O(n)$ auxiliary space — Accounted for by the recursive call stack depth of at most n and the subset array storing up to n elements.

Q3. Graph Coloring Problem Develop a pseudocode using Backtracking to assign colors to vertices of a graph such that no two adjacent vertices share the same color. Clearly identify inputs (graph, number of colors) and output (valid coloring). Design a correct and efficient algorithm using loops and condition checks. Ensure proper use of recursion and validation functions. Structure the pseudocode clearly and logically. Handle all constraints and ensure completeness of the solution.

Problem Definition

Given an undirected graph $G = (V, E)$ and m colors, assign a color to every vertex $v \in V$ such that no two adjacent vertices share the same color (m -coloring problem).

Input: Graph G represented as an adjacency matrix $graph[V][V]$, number of vertices V , and available colors count m .

Output: Array $color[0..V-1]$ mapping each vertex to a color index (1 to m), or FALSE if m -coloring is impossible.

Algorithmic Approach

We use recursive backtracking to assign colors vertex-by-vertex (from vertex 0 to $V-1$). For vertex u , we try assigning color c from 1 to m . Before assignment, a validation step checks if any adjacent vertex already shares color c . If safe, we recursive to vertex $u + 1$. If no color from 1 to m works, we backtrack to vertex $u - 1$.

State-Space Representation

An m -ary state-space tree of depth $V + 1$. Each node at level u represents color assignment choices for vertex u . Branching factor is m (colors 1 to m).

Pseudocode

```
Algorithm SolveGraphColoring(graph, V, m):
    Input: Adjacency matrix graph[V][V], vertex count V, available colors m
    Output: Color placement array or FALSE

    Initialize color[0..V-1] with 0    // 0 means uncolored

    if GraphColoringRecursive(graph, V, m, color, 0) ==
TRUE:        Print "Solution Found: ", color        return
TRUE    else:
        Print "No solution exists with ", m, " colors"
    return FALSE

Algorithm GraphColoringRecursive(graph, V, m, color, u):
// Base Case: All vertices are successfully colored
if u == V:
    return TRUE

    // Try all colors from 1 to m for vertex u
for c = 1 to m:
    if IsColorSafe(graph, V, color, u, c) == TRUE:
        color[u] = c    // Assign color c to vertex u

        // Recursive Call: Color next vertex        if
        GraphColoringRecursive(graph, V, m, color, u + 1) == TRUE:
            return TRUE

        // Backtracking Step: Undo assignment
        color[u] = 0

    return FALSE    // Backtrack to previous vertex

Algorithm IsColorSafe(graph, V, color, u, c):
for v = 0 to V - 1:
    // Check if vertex v is adjacent to u and already has color c
    if graph[u][v] == 1 and color[v] == c:
        return FALSE
return TRUE
```

Logical Explanation

1. Vertex-by-vertex coloring: Process vertices sequentially in order 0, 1, ..., V-1.
2. Color Loop: Tries each color c from 1 to m.
3. IsColorSafe(): Scans row graph[u] to ensure no neighbor v (where graph[u][v] == 1) currently shares color c.
4. Recursive Step & Backtrack: If safe, assigns color[u] = c and moves to u + 1. Reverts color[u] = 0 if child calls fail.

Complexity Analysis

Worst-case Time Complexity: $O(m^V)$ — In the worst case, trying m colors across V vertices yields total operations bounded by m^V .

Auxiliary Space Complexity: $O(V)$ recursion depth + color array space.

Graph Storage Complexity: $O(V^2)$ space required to store graph connectivity when using an Adjacency Matrix format (or $O(V + E)$ if an Adjacency List is used). Total Space = $O(V^2) + O(V)$.

Code of Conduct

I V Deepa Dharshini (192525213) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

KS THARUNESHWAR

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
Overall Total (100)						

Faculty In-charge

Course Coordinator

Head of Department

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____